



FOUR-COURSE SYSTEM

COURSE 3 - DATA ENGINEERING 2026

C3-050 Beginner-First Lesson Book

13 storage/lakehouse lessons rebuilt into the full M01 teacher loop.

Artifact	C3-050
Status	2026 - LEARNER EDITION
Phase	C3-04 Storage, File Formats & Lakehouse 13 lessons 34-46 focused hours
Prerequisite	C3-03 Gate PASS; Bridge warehousing/lakehouse mental models assumed, implementation depth taught here.
Brand	Charcoal #1F2937 Teal #14B8A6 Soft Blue #3B82F6 AMOLED #000

House teaching rule: mental picture -> tiny example -> guided real file/table workflow -> break/fix -> independent changed task -> validation -> production reasoning -> evidence. A file extension or successful write is never sufficient proof of correct table state.

This book is your teacher

Use this book as the primary teacher. External videos and official documentation are visual or current-behavior companions, never hidden prerequisites.

The locked beginner-first lesson contract

- | | |
|-----------------------------------|---|
| 01. Why it matters | 09. Why it worked |
| 02. What you learn | 10. Common beginner mistakes |
| 03. Prerequisite / starting state | 11. If stuck / recovery |
| 04. Picture it in your head | 12. Small independent task |
| 05. New words before use | 13. Validate |
| 06. Useful video/source | 14. Teach it back |
| 07. Follow with me once | 15. Protected review + changed retry |
| 08. Expected result | 16. Evidence/mastery + exact next route |

Rules that protect your learning

- Save a genuine attempt before opening a protected review.
- After review, close it and solve a changed retry - do not copy the reviewed answer.
- A command that runs is not mastery: validate grain, keys, counts, state, failure/recovery and evidence.
- If a new engineering concept appears, this book explains it from first principles even when Bridge skills are assumed.
- When stuck, repair only the named weak competency; do not restart the whole phase.

Contents / Index

Actual page numbers after the master-audit repair. Mentor routes use these same pages.

Front matter	Page
This book is your teacher	2
Contents / Index	3
Module map + practice/review/Gate route	4
Phase-specific study notes	5
Lessons	Start
C3-04-L01 Object storage and namespace mental model	6
C3-04-L02 CSV, JSON, Avro and Parquet trade-offs	9
C3-04-L03 Columnar storage, statistics and compression	12
C3-04-L04 Partitioning strategy and pruning	15
C3-04-L05 Small-files problem and compaction	18
C3-04-L06 Catalogs, metadata and table namespaces	21
C3-04-L07 Lake vs warehouse vs lakehouse + open table-format trade-offs	24
C3-04-L08 Delta transaction log and ACID guarantees	27
C3-04-L09 Schema enforcement and schema evolution	30
C3-04-L10 MERGE, upsert, update and delete patterns	33
C3-04-L11 Time travel and reproducible historical reads	36
C3-04-L12 Medallion/layered architecture and table contracts	39
C3-04-L13 Optimization, retention and maintenance strategy	42
Mastery checklist + revision quick reference	45

Module map + protected learning route

Storage, File Formats & Lakehouse

- 01 Prerequisite / prior-phase Gate
- 02 Lesson mental model + vocabulary
- 03 Follow guided example once
- 04 Matching independent practice
- 05 Save genuine attempt
- 06 Protected review unlocks
- 07 Close review + changed retry
- 08 Validate + explain back
- 09 Master lesson + save evidence
- 10 Mini-Lab / integrated drill
- 11 Protected Gate A
- 12 If needed: protected review -> targeted repair -> changed micro-proof
- 13 Explicitly assign fresh Gate B/C only after repair evidence
- 14 Phase PASS -> next phase

What counts as mastery

- You can rebuild the competency without the lesson/review open.
- Your validation catches a plausible wrong result, not only syntax failure.
- You can state one failure mode and the recovery step.
- Your evidence names the input/output or artifact and the independent check used.

Revision rule

Revise from evidence: weak validation, weak recovery or weak explanation sends you back only to the matching lesson/repair route. Completion percentage never overrides a failed Gate.



How to use this book

Each lesson has three pages: A = why/outcome/prerequisite/picture/vocabulary/source; B = guided example/result/debug; C = independent transfer/recovery/validation/mastery. Videos are optional visual reinforcement; the book is sufficient fallback.



C3-04-L01 - Object storage and namespace mental model

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Poor namespace design makes reprocessing, retention, migration and ownership harder and encourages applications to depend on internal file names.

WHAT YOU LEARN

Separate physical object locations from logical table identity and design stable namespaces that survive file maintenance.

PREREQUISITE

C3-03 Gate PASS. Bridge warehousing/lakehouse concepts are assumed. Within C3-04, previous lesson mastery/remediation is expected. Parquet and transactional-table mechanics are taught from first principles here.

PICTURE IT IN YOUR HEAD

Picture a city address system. The street address is physical; the business name in the directory is logical. Files may move or be compacted while the table name consumers use should remain stable.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
object	Bytes stored under a key/URI in object storage.
key/URI	Physical address used to locate an object.
namespace	Naming structure for domains, layers and tables.
logical table	Stable table identity independent of individual data-file names.

USEFUL SOURCE / VIDEO

RECOMMENDED - [AWS S3 object model](#)

Read object/key/metadata concepts. Purpose: reinforce object storage vs local filesystem; vendor specifics remain secondary.



C3-04-L01 - Follow with me once

Page B - engineering idea, tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

Object storage is key-addressed storage, not a giant local drive. A bucket/container holds objects addressed by keys/URIs. Rename/move semantics may be implemented as copy/delete, and a logical table name should not be coupled to one transient file name. Think in layers: object store -> files -> table metadata -> catalog -> consuming engine.

TINY WORKED EXAMPLE

Map raw/orders/2026/09/ to a physical landing location, then map catalog.analytics.orders to a stable logical table identity. Mark which names may change during compaction and which contract should remain stable.

FOLLOW WITH ME ONCE

Draw physical URI, table metadata location and logical table name as three distinct boxes. Add one retention boundary and one ownership boundary.

EXPECTED RESULT

No generated part-file name is a business contract; raw evidence is separated from curated state; logical names remain stable.

WHY IT WORKED

Poor namespace design makes reprocessing, retention, migration and ownership harder and encourages applications to depend on internal file names.

BREAK / FIX / DEBUG

If every consumer points directly to a generated file, introduce a stable table/catalog boundary before adding more data.



C3-04-L01 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Treating object storage like a local disk with reliable atomic rename.
- Letting dashboards depend on generated part-file names.
- Mixing raw evidence and curated tables in one ambiguous path.
- Using paths as the only ownership/contract metadata.

STUCK? RECOVERY

1) What is physical? 2) What is logical? 3) What changes during maintenance? 4) What must consumers never depend on?

SMALL INDEPENDENT TASK

Design the namespace for a second domain (support tickets) using different physical locations but the same naming principles.

VALIDATE

No generated part-file name is a business contract; raw evidence is separated from curated state; logical names remain stable.

THEN SHOW THE PROFESSIONAL VERSION

Defend a fresh namespace design and identify one migration/maintenance benefit.

TEACH IT BACK

Why is object storage not equivalent to a local filesystem?

PROTECTED REVIEW + CHANGED RETRY

Save P01 first. Only then open R01. Close review and solve the changed version without reopening the answer. The original attempt remains in history.

EVIDENCE / MASTERY

Namespace map + ownership/retention note + fresh transfer saved.

MASTER only after the independent changed version passes validation and evidence is saved.



C3-04-L02 - CSV, JSON, Avro and Parquet trade-offs

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Using text files for every layer increases ambiguity and scan cost; using Parquet for every interchange boundary can reduce debuggability and compatibility.

WHAT YOU LEARN

Choose an interchange or analytical file format from schema, access pattern, interoperability, compression and operational needs.

PREREQUISITE

C3-03 Gate PASS. Bridge warehousing/lakehouse concepts are assumed. Within C3-04, previous lesson mastery/remediation is expected. Parquet and transactional-table mechanics are taught from first principles here.

PICTURE IT IN YOUR HEAD

Picture four shipping containers. CSV is a simple open box, JSON carries nested labeled parcels, Avro is row-oriented with schema, and Parquet stacks values by column for analytical scanning. The best box depends on the trip.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
row-oriented	Values for one record are stored together.
columnar	Values from the same column are stored together.
schema	Defined field names/types/structure.
interchange	Moving data between systems rather than optimizing analytics.

USEFUL SOURCE / VIDEO

RECOMMENDED - [Apache Parquet overview](#)

Use the official column-oriented purpose/format overview; compare against CSV/JSON/Avro needs rather than declaring a universal winner.

OPTIONAL - [Apache Avro specification](#)

Use schema + row-oriented serialization sections when comparing interchange/event use cases.



C3-04-L02 - Follow with me once

Page B - engineering idea, tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

CSV and JSON are convenient boundaries but weak analytical storage contracts. Avro is row-oriented and schema-bearing, useful for event/message ecosystems. Parquet is typed, columnar and optimized for analytical scans. No format is universally best: choose by workload and lifecycle.

TINY WORKED EXAMPLE

Compare four workloads: analyst export, nested API landing, schema-bearing event archive, and 500M-row analytical history. State the winning format and the main trade-off for each.

FOLLOW WITH ME ONCE

Build a six-column decision matrix and score each candidate against the workload rather than against marketing claims.

EXPECTED RESULT

Decision mentions schema strength, access pattern and consumer/tool compatibility; trade-offs are explicit.

WHY IT WORKED

Using text files for every layer increases ambiguity and scan cost; using Parquet for every interchange boundary can reduce debuggability and compatibility.

BREAK / FIX / DEBUG

If your answer is based only on file size or popularity, restate the access pattern and schema/consumer requirements.



C3-04-L02 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Saying Parquet is always best.
- Using CSV for nested event payloads without acknowledging loss/flattening.
- Choosing a format without naming the consumers/query pattern.
- Confusing Parquet file format with Delta/Iceberg table format.

STUCK? RECOVERY

1) Exchange or analytics? 2) Row or column access? 3) Schema contract? 4) Nested data? 5) Human inspection?

SMALL INDEPENDENT TASK

Choose a format for a mobile-app event archive with nested attributes and a downstream warehouse load.

VALIDATE

Decision mentions schema strength, access pattern and consumer/tool compatibility; trade-offs are explicit.

THEN SHOW THE PROFESSIONAL VERSION

Make a defensible fresh file-format decision with explicit assumptions.

TEACH IT BACK

When is CSV acceptable even though Parquet is more efficient for analytics?

PROTECTED REVIEW + CHANGED RETRY

Save P02 first. Only then open R02. Close review and solve the changed version without reopening the answer. The original attempt remains in history.

EVIDENCE / MASTERY

Decision matrix + changed scenario saved.

MASTER only after the independent changed version passes validation and evidence is saved.



C3-04-L03 - Columnar storage, statistics and compression

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

A .parquet extension is not proof of efficient analytical storage. Engineers must understand and inspect the physical metadata that query engines use.

WHAT YOU LEARN

Create and inspect Parquet, connect row groups/column chunks/statistics to projection and selective scans, and validate row/schema parity.

PREREQUISITE

C3-03 Gate PASS. Bridge warehousing/lakehouse concepts are assumed. Within C3-04, previous lesson mastery/remediation is expected. Parquet and transactional-table mechanics are taught from first principles here.

PICTURE IT IN YOUR HEAD

Picture a library that stores all values from the same column together. A query asking only for region and amount can visit those shelves and skip unrelated shelves; metadata can also help it skip whole shelf sections.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
row group	A horizontal chunk of rows in a Parquet file.
column chunk	Data for one column inside a row group.
statistics	Metadata such as min/max/null counts that can help skipping.
projection	Reading only selected columns.

USEFUL SOURCE / VIDEO

REQUIRED - [Apache Arrow - Reading/Writing Parquet](#)

Follow single-file write/read, metadata inspection, projection and compression sections.

RECOMMENDED - [Apache Parquet file format](#)

Use row-group/column-chunk/file-metadata structure to connect physical layout to engine reads.



C3-04-L03 - Follow with me once

Page B - engineering idea, tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

Parquet organizes column chunks inside row groups. Engines can project only required columns and may use metadata such as min/max/null counts to skip work. Compression and encoding exploit repeated/similar values, but physical layout and row-group size still matter.

TINY WORKED EXAMPLE

Convert the supplied orders CSV to Parquet, inspect schema and row-group metadata, then read only order_id, region and amount. Compare projected columns with a full read while preserving row count.

FOLLOW WITH ME ONCE

Read CSV -> explicit Arrow types where needed -> write Parquet -> reopen ParquetFile -> inspect metadata -> projected read -> reconcile.

```
import pyarrow.csv as csv
import pyarrow.parquet as pq
t = csv.read_csv('data/orders_day1.csv')
pq.write_table(t, 'out/orders.parquet')
pf = pq.ParquetFile('out/orders.parquet')
print(pf.metadata.num_rows, pf.metadata.num_row_groups)
print(pq.read_table('out/orders.parquet', columns=['order_id', 'region']))
```

EXPECTED RESULT

Source and Parquet logical rows match; required fields/types are intentional; projected read returns only requested columns.

WHY IT WORKED

A .parquet extension is not proof of efficient analytical storage. Engineers must understand and inspect the physical metadata that query engines use.

BREAK / FIX / DEBUG

If PyArrow is unavailable, run 00_runtime_check.py, install the pinned requirements in the course environment, then rerun. Do not fabricate metadata output.



C3-04-L03 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Trusting the .parquet extension without reading metadata.
- Assuming columnar automatically means fast for every query.
- Ignoring row-group/statistics layout.
- Comparing file size only while forgetting row/schema parity.

STUCK? RECOVERY

1) Rows preserved? 2) Schema correct? 3) Row groups? 4) Projection? 5) Statistics available?

SMALL INDEPENDENT TASK

Create a fresh Parquet file from customers_day1.csv and inspect a different subset of columns.

VALIDATE

Source and Parquet logical rows match; required fields/types are intentional; projected read returns only requested columns.

THEN SHOW THE PROFESSIONAL VERSION

Build and inspect a fresh Parquet dataset without relying on copied output.

TEACH IT BACK

How can columnar layout reduce I/O for a query selecting 3 of 40 columns?

PROTECTED REVIEW + CHANGED RETRY

Save P03 first. Only then open R03. Close review and solve the changed version without reopening the answer. The original attempt remains in history.

EVIDENCE / MASTERY

Runnable scripts + real metadata output + parity checks.

MASTER only after the independent changed version passes validation and evidence is saved.

C3-04-L04 - Partitioning strategy and pruning

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Bad partitioning turns a correct table into an expensive metadata problem and can make small-file pressure worse.

WHAT YOU LEARN

Choose partition keys from query patterns and cardinality, then prove the partition plan does not create pathological file counts.

PREREQUISITE

C3-03 Gate PASS. Bridge warehousing/lakehouse concepts are assumed. Within C3-04, previous lesson mastery/remediation is expected. Parquet and transactional-table mechanics are taught from first principles here.

PICTURE IT IN YOUR HEAD

Picture a warehouse split into aisles by a useful label such as date. If workers usually ask for one date, they can skip other aisles. If every customer gets an aisle, you create millions of tiny aisles and waste time.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
partition	Logical/physical subdivision of a dataset by values.
pruning	Skipping partitions/files that cannot match a filter.
cardinality	Number of distinct values in a field.
partition explosion	Too many tiny partitions/files from a poor key.

USEFUL SOURCE / VIDEO

REQUIRED - [Apache Arrow Parquet datasets](#)

Use Partitioned Datasets section: directory partitions, dataset read/write, and filter/pruning reasoning.



C3-04-L04 - Follow with me once

Page B - engineering idea, tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

Partitioning can eliminate whole file groups before deeper scanning when filters align with partition values. It is not an index. High-cardinality keys can create thousands of tiny directories/files, and partitioning on a field rarely filtered by consumers can add cost without benefit.

TINY WORKED EXAMPLE

Compare region, event_date and customer_id for the practice data. Region has low cardinality; event_date can be useful for bounded history; customer_id is a high-cardinality trap for this workload.

FOLLOW WITH ME ONCE

Query pattern -> candidate keys -> cardinality -> expected rows/files per partition -> choose or reject.

EXPECTED RESULT

Partition key is tied to actual filters; expected partition cardinality and small-file risk are documented.

WHY IT WORKED

Bad partitioning turns a correct table into an expensive metadata problem and can make small-file pressure worse.

BREAK / FIX / DEBUG

If you picked a key only because it exists, list the top three real filter predicates and estimate cardinality first.



C3-04-L04 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Partitioning by a high-cardinality ID.
- Choosing a partition key only because it exists.
- Claiming partitioning is the same as an index.
- Ignoring file count/size per partition and real filter predicates.

STUCK? RECOVERY

1) Frequent filter? 2) Cardinality? 3) Data volume per value? 4) Lifecycle? 5) File-size health?

SMALL INDEPENDENT TASK

Re-evaluate the plan for a workload that queries mostly by service_date rather than region.

VALIDATE

Partition key is tied to actual filters; expected partition cardinality and small-file risk are documented.

THEN SHOW THE PROFESSIONAL VERSION

Defend a fresh partition design and reject a high-cardinality trap.

TEACH IT BACK

Why is partitioning not the same as a database index?

PROTECTED REVIEW + CHANGED RETRY

Save P04 first. Only then open R04. Close review and solve the changed version without reopening the answer. The original attempt remains in history.

EVIDENCE / MASTERY

Decision + partition plan + changed-query transfer.

MASTER only after the independent changed version passes validation and evidence is saved.



C3-04-L05 - Small-files problem and compaction

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

A correct dataset can still be operationally unhealthy; conversely, rewriting files every day can waste compute and increase risk.

WHAT YOU LEARN

Measure file-layout health and define a compaction trigger that preserves logical records and does not run blindly.

PREREQUISITE

C3-03 Gate PASS. Bridge warehousing/lakehouse concepts are assumed. Within C3-04, previous lesson mastery/remediation is expected. Parquet and transactional-table mechanics are taught from first principles here.

PICTURE IT IN YOUR HEAD

Picture a desk covered by ten thousand tiny envelopes. The information is correct, but opening/listing each envelope has overhead. Compaction combines envelopes into healthy parcels without changing the letters inside.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
small-file problem	Excessive tiny files causing listing/planning/read overhead.
compaction	Rewriting many small files into fewer healthier files.
file health	Metrics such as file count/size/skew by partition.
parity check	Proof business rows/keys/metrics did not change after rewrite.

USEFUL SOURCE / VIDEO

REQUIRED - [delta-rs small-file compaction](#)

Use file-count cause, optimize/compaction and vacuum-after-optimize distinction; course still requires measurable trigger + parity controls.



C3-04-L05 - Follow with me once

Page B - engineering idea, tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

Many tiny files create listing, metadata, planning and scheduling overhead. Compaction rewrites physical files into healthier files without changing business records. File count, median/average size, partition skew and read patterns should drive the decision.

TINY WORKED EXAMPLE

Use the supplied synthetic file-size manifest to compare 48 tiny files against a compacted target. Preserve rows/key uniqueness before and after the proposed rewrite.

FOLLOW WITH ME ONCE

Measure -> compare to target -> estimate rewrite scope -> compact only affected partitions -> reconcile rows/keys/metrics.

EXPECTED RESULT

Compaction trigger has metrics and thresholds; post-rewrite parity checks are explicit.

WHY IT WORKED

A correct dataset can still be operationally unhealthy; conversely, rewriting files every day can waste compute and increase risk.

BREAK / FIX / DEBUG

If your rule says "OPTIMIZE daily," replace the calendar rule with measurable file-health thresholds.



C3-04-L05 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Running compaction on a calendar with no file-health trigger.
- Measuring only total bytes, not file count/size distribution.
- Skipping post-compaction row/key/metric reconciliation.
- Assuming optimize and vacuum are the same operation.

STUCK? RECOVERY

1) File count? 2) Size distribution? 3) Partition skew? 4) Read cost? 5) Parity check?

SMALL INDEPENDENT TASK

Design a trigger for a table with fewer files but extreme size skew across regions.

VALIDATE

Compaction trigger has metrics and thresholds; post-rewrite parity checks are explicit.

THEN SHOW THE PROFESSIONAL VERSION

Diagnose a fresh small-file pattern and define a safe compaction plan.

TEACH IT BACK

Why can a table be logically correct but physically unhealthy?

PROTECTED REVIEW + CHANGED RETRY

Save P05 first. Only then open R05. Close review and solve the changed version without reopening the answer. The original attempt remains in history.

EVIDENCE / MASTERY

Health evidence + trigger + parity plan.

MASTER only after the independent changed version passes validation and evidence is saved.



C3-04-L06 - Catalogs, metadata and table namespaces

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Without metadata ownership and contracts, teams cannot discover, trust, migrate or govern tables consistently.

WHAT YOU LEARN

Register a table as a governed logical object with owner, schema, grain, location, partitioning, freshness, quality and retention metadata.

PREREQUISITE

C3-03 Gate PASS. Bridge warehousing/lakehouse concepts are assumed. Within C3-04, previous lesson mastery/remediation is expected. Parquet and transactional-table mechanics are taught from first principles here.

PICTURE IT IN YOUR HEAD

Picture a card catalog. Object paths tell you where bytes live; the catalog tells people what table those bytes represent, its schema/grain/owner, and how to find it without memorizing file names.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
catalog	Service/metadata layer resolving logical tables and properties.
metadata	Information about schema, files, versions, owner and table properties.
namespace	Logical grouping for tables, often catalog/schema/table.
ownership	Named responsibility for table contract and change decisions.

USEFUL SOURCE / VIDEO

RECOMMENDED - [Apache Iceberg catalogs](#)

Use catalog/warehouse/REST-catalog concepts to reinforce logical table identity and metadata boundary.



C3-04-L06 - Follow with me once

Page B - engineering idea, tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

A folder of Parquet files is not automatically a governed table. A catalog/metadata layer provides stable names, schemas, locations, ownership, discovery and often access controls. Even in a local course project, writing the registration contract teaches the same operational boundary.

TINY WORKED EXAMPLE

Create a registration record for analytics.orders_current with physical location, grain one row per order_id, owner, required columns, partitioning, refresh cadence, quality checks and retention.

FOLLOW WITH ME ONCE

Name -> purpose -> grain -> keys -> schema -> physical location -> partitions -> freshness -> quality -> owner -> retention.

EXPECTED RESULT

Contract distinguishes logical identity from physical path and includes operational ownership.

WHY IT WORKED

Without metadata ownership and contracts, teams cannot discover, trust, migrate or govern tables consistently.

BREAK / FIX / DEBUG

If the contract is only a path and column list, add purpose, grain, owner, quality, refresh and lifecycle.



C3-04-L06 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Calling a folder of files a governed table.
- Recording only path and columns, not grain/owner/quality/lifecycle.
- Embedding credentials in metadata contracts.
- Changing physical location without a stable logical identity.

STUCK? RECOVERY

1) Name? 2) Grain? 3) Keys? 4) Owner? 5) Quality/freshness? 6) Retention?

SMALL INDEPENDENT TASK

Write a registration record for support.tickets_current with different ownership and retention.

VALIDATE

Contract distinguishes logical identity from physical path and includes operational ownership.

THEN SHOW THE PROFESSIONAL VERSION

Write and defend a fresh table-registration contract.

TEACH IT BACK

What does a catalog add beyond a directory of files?

PROTECTED REVIEW + CHANGED RETRY

Save P06 first. Only then open R06. Close review and solve the changed version without reopening the answer. The original attempt remains in history.

EVIDENCE / MASTERY

Completed contract + fresh transfer.

MASTER only after the independent changed version passes validation and evidence is saved.



C3-04-L07 - Lake vs warehouse vs lakehouse + open table-format trade-offs

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Technology selection based on logos or "open" marketing creates hidden integration cost. Engineering choices need workload and ecosystem evidence.

WHAT YOU LEARN

Compare lake, warehouse and lakehouse architectures and make a conditional Delta-vs-Iceberg decision without declaring a universal winner.

PREREQUISITE

C3-03 Gate PASS. Bridge warehousing/lakehouse concepts are assumed. Within C3-04, previous lesson mastery/remediation is expected. Parquet and transactional-table mechanics are taught from first principles here.

PICTURE IT IN YOUR HEAD

Picture three layers: files on storage, table metadata above them, and query engines above that. Parquet is a file format; Delta/Iceberg are table formats that track versions/metadata over files; a lakehouse combines open storage with table semantics.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
data lake	Open-file/object-storage oriented analytical storage.
warehouse	Managed analytical database/table environment.
lakehouse	Open storage plus reliable table semantics/management.
open table format	Specification/metadata system that manages files as versioned tables.

USEFUL SOURCE / VIDEO

REQUIRED - [Delta Lake current project documentation](#)

Use current lakehouse/ACID/versioning capabilities; project release context is refreshed separately from the local delta-rs lab.

REQUIRED - [Apache Iceberg latest documentation](#)

Use open-table-format, schema/partition evolution and multi-engine concepts.

RECOMMENDED - [Apache Iceberg table spec](#)

Use format-version and metadata/snapshot model for deeper comparison.



C3-04-L07 - Follow with me once

Page B - engineering idea, tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

A lake stores open files; a warehouse provides managed relational/analytical semantics; a lakehouse adds table semantics over open object-storage data. Delta and Iceberg both provide transactional/versioned table metadata above data files, but ecosystem integrations and metadata models differ. Current Iceberg 1.11.0 adds mature multi-engine capabilities; current Delta 4.4.0 continues its catalog-managed and multi-engine evolution.

TINY WORKED EXAMPLE

Compare a Fabric/Delta-centered team with a Spark/Trino/Flink multi-engine platform. State assumptions, required catalog/runtime evidence and migration/interoperability needs.

FOLLOW WITH ME ONCE

Workload -> engines -> catalog -> DML needs -> schema/partition evolution -> operations -> interoperability -> decision.

EXPECTED RESULT

Decision is conditional, evidence-based and separates table-format semantics from storage file format.

WHY IT WORKED

Technology selection based on logos or "open" marketing creates hidden integration cost. Engineering choices need workload and ecosystem evidence.

BREAK / FIX / DEBUG

If your decision says one format is always better, add platform, engine, governance and migration assumptions.



C3-04-L07 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Declaring Delta or Iceberg universally better.
- Comparing logos instead of engines/catalog/DML/operations.
- Treating Parquet, Delta and Iceberg as interchangeable.
- Ignoring migration/interoperability needs.

STUCK? RECOVERY

1) Engines? 2) Catalog? 3) DML? 4) Evolution? 5) Ops? 6) Interop?

SMALL INDEPENDENT TASK

Evaluate a second scenario where the primary query engine changes from Spark to Trino.

VALIDATE

Decision is conditional, evidence-based and separates table-format semantics from storage file format.

THEN SHOW THE PROFESSIONAL VERSION

Defend a fresh open-table-format choice with explicit assumptions and reversal conditions.

TEACH IT BACK

How are Parquet, Delta Lake and Iceberg related but not interchangeable concepts?

PROTECTED REVIEW + CHANGED RETRY

Save P07 first. Only then open R07. Close review and solve the changed version without reopening the answer. The original attempt remains in history.

EVIDENCE / MASTERY

Decision matrix + changed-engine transfer.

MASTER only after the independent changed version passes validation and evidence is saved.



C3-04-L08 - Delta transaction log and ACID guarantees

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Without transactional table state, readers/writers can observe inconsistent file sets during change and recovery.

WHAT YOU LEARN

Create a local Delta table, inspect committed version state, and explain why the table is more than the current Parquet file set.

PREREQUISITE

C3-03 Gate PASS. Bridge warehousing/lakehouse concepts are assumed. Within C3-04, previous lesson mastery/remediation is expected. Parquet and transactional-table mechanics are taught from first principles here.

PICTURE IT IN YOUR HEAD

Picture a bank ledger. New data files alone do not define account truth; a committed transaction record tells readers which files/actions belong to version N. The Delta log is the ledger of committed table state.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
transaction log	Ordered record of table actions/commits.
ACID	Atomicity, consistency, isolation and durability properties of transactions.
snapshot	Committed view of table state at one version.
commit	Atomic publication of a new table version.

USEFUL SOURCE / VIDEO

REQUIRED - [Delta Lake documentation](#)

Use ACID, scalable metadata, transaction-log/versioned-table concepts.

RECOMMENDED VIDEO - [Databricks - Diving into Delta Lake: Unpacking the Transaction Log](#)

Approx. 30 min. Focus: log actions, atomicity/serializability, checkpoints, state reconstruction and time travel. Use topic chapters; external media is optional.



C3-04-L08 - Follow with me once

Page B - engineering idea, tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

Delta records table actions in a transaction log so readers reconstruct a committed table version rather than consume an arbitrary halfway file set. ACID semantics concern committed table state, not merely the existence of Parquet files. The local lab uses delta-rs/deltalake 1.6.3.

TINY WORKED EXAMPLE

Create trusted/orders as a Delta table from day-1 rows. Inspect `dt.version()`, schema, logical row count and the `_delta_log` directory. Identify data files separately from metadata/commit files.

FOLLOW WITH ME ONCE

Read typed data -> write Delta -> reopen DeltaTable -> version/schema/rows -> inspect `_delta_log` -> reconcile.

```
from deltalake import DeltaTable, write_deltalake
write_deltalake('out/orders_delta', arrow_table, mode='overwrite')
dt = DeltaTable('out/orders_delta')
print('version', dt.version())
print(dt.schema())
```

EXPECTED RESULT

Committed version observed; row/key parity proven; no "success" claim from file existence alone.

WHY IT WORKED

Without transactional table state, readers/writers can observe inconsistent file sets during change and recovery.

BREAK / FIX / DEBUG

Missing dependency -> install pinned requirements. Runtime error -> save the actual error and inspect version/path/schema assumptions before retrying.



C3-04-L08 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Calling any folder of Parquet files a Delta table.
- Using file existence instead of committed version evidence.
- Editing `_delta_log` files manually.
- Claiming ACID without observing a real table version/log state.

STUCK? RECOVERY

1) Table path? 2) Version? 3) Schema? 4) Rows? 5) `_delta_log`? 6) Reconcile?

SMALL INDEPENDENT TASK

Create a fresh Delta table for `customers_day1.csv` at a different path.

VALIDATE

Committed version observed; row/key parity proven; no "success" claim from file existence alone.

THEN SHOW THE PROFESSIONAL VERSION

Create and inspect a fresh local Delta table and explain the committed-state model.

TEACH IT BACK

Why is the transaction log central to atomic table state?

PROTECTED REVIEW + CHANGED RETRY

Save P08 first. Only then open R08. Close review and solve the changed version without reopening the answer. The original attempt remains in history.

EVIDENCE / MASTERY

Real version/schema/count/log evidence + fresh transfer.

MASTER only after the independent changed version passes validation and evidence is saved.



C3-04-L09 - Schema enforcement and schema evolution

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Silent schema drift can break readers, corrupt metrics or create ambiguous null behavior long after the write succeeded.

WHAT YOU LEARN

Treat schema change as a controlled contract event: block incompatible writes and deliberately permit supported additive evolution.

PREREQUISITE

C3-03 Gate PASS. Bridge warehousing/lakehouse concepts are assumed. Within C3-04, previous lesson mastery/remediation is expected. Parquet and transactional-table mechanics are taught from first principles here.

PICTURE IT IN YOUR HEAD

Picture a building entry policy. Schema enforcement rejects an invalid visitor; schema evolution is an approved change to the guest list. Evolution is controlled change, not "accept anything."

NEW WORDS BEFORE WE USE THEM

Word	Meaning
enforcement	Rejecting writes incompatible with current schema/constraints.
evolution	Deliberately changing the table schema.
additive change	Adding a field while preserving existing fields.
breaking change	Change likely to invalidate existing readers/contracts.

USEFUL SOURCE / VIDEO

REQUIRED - [delta-rs - Writing Delta Tables](#)

Use schema_mode overwrite/merge behavior and the warning that schema mismatch fails by default.



C3-04-L09 - Follow with me once

Page B - engineering idea, tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

Schema enforcement protects a table from incompatible writes. Evolution is not "accept whatever arrives"; it is an owned change with compatibility reasoning. Adding a nullable field is typically less disruptive than rename/drop/type narrowing, but every change needs validation and downstream impact analysis.

TINY WORKED EXAMPLE

Day 2 adds channel. First classify the change, then apply it deliberately to the trusted table. Contrast this with renaming amount to net_amount or changing amount from numeric to free text.

FOLLOW WITH ME ONCE

Detect diff -> classify additive/rename/drop/type -> owner/compatibility -> apply allowed change -> reopen schema -> validate readers.

```
# New nullable field is allowed only after contract decision.
write_deltalake(path, day2_table, mode='append', schema_mode='merge')
# Reopen and validate the exact new schema; do not use permissive mode blindly.
```

EXPECTED RESULT

Schema diff recorded; change approval explicit; downstream/read compatibility considered.

WHY IT WORKED

Silent schema drift can break readers, corrupt metrics or create ambiguous null behavior long after the write succeeded.

BREAK / FIX / DEBUG

If the write failed, inspect whether the runtime needs explicit schema mode/evolution behavior; do not bypass the contract with broad permissive settings.



C3-04-L09 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Turning on permissive evolution for every write.
- Treating a new nullable column as automatically harmless.
- Renaming/dropping keys without downstream impact analysis.
- Failing to save old/new schema evidence.

STUCK? RECOVERY

1) What changed? 2) Compatible? 3) Owner? 4) Runtime option? 5) Downstream check?

SMALL INDEPENDENT TASK

Classify a fresh change that widens a numeric type and another that renames a required key.

VALIDATE

Schema diff recorded; change approval explicit; downstream/read compatibility considered.

THEN SHOW THE PROFESSIONAL VERSION

Handle a fresh schema change without silently weakening the table contract.

TEACH IT BACK

What is the difference between schema enforcement and schema evolution?

PROTECTED REVIEW + CHANGED RETRY

Save P09 first. Only then open R09. Close review and solve the changed version without reopening the answer. The original attempt remains in history.

EVIDENCE / MASTERY

Runtime/schema evidence + contract change + fresh classification.

MASTER only after the independent changed version passes validation and evidence is saved.



C3-04-L10 - MERGE, upsert, update and delete patterns

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Appending corrections creates duplicate current-state entities; an unsafe merge predicate can update/delete too much.

WHAT YOU LEARN

Apply current-state corrections with a grain-safe merge predicate, deterministic source rows and before/after reconciliation.

PREREQUISITE

C3-03 Gate PASS. Bridge warehousing/lakehouse concepts are assumed. Within C3-04, previous lesson mastery/remediation is expected. Parquet and transactional-table mechanics are taught from first principles here.

PICTURE IT IN YOUR HEAD

Picture correcting a customer record. Append makes a second current row; MERGE can update the matched key and insert only unmatched keys in one table transaction. The key/predicate decides whether the correction is safe.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
MERGE	Conditional matched/unmatched table transaction.
upsert	Update matching keys and insert new keys.
predicate	Condition matching source rows to target rows.
idempotent	Rerunning the same logical change converges to the same intended state.

USEFUL SOURCE / VIDEO

REQUIRED - [delta-rs - Merging a Table](#)

Use `dt.merge/TableMerger` matched-update + unmatched-insert patterns and merge performance notes.



C3-04-L10 - Follow with me once

Page B - engineering idea, tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

MERGE separates matched updates from unmatched inserts inside one transactional operation. The predicate must match target grain, and the source must not contain ambiguous multiple versions for one target key. Operation metrics are evidence, but output integrity still needs independent key/count/measure checks.

TINY WORKED EXAMPLE

Day 2 updates O1002 and inserts O1007. Prove one source winner per order_id, merge on order_id, then verify one O1002 row with the new values and one O1007 row.

FOLLOW WITH ME ONCE

Validate source -> open target -> merge matched update/unmatched insert -> reopen -> reconcile keys/counts/amounts.

```
dt.merge(source=source, predicate='target.order_id = source.order_id',
         source_alias='source', target_alias='target')\
    .when_matched_update_all()\
    .when_not_matched_insert_all()\
    .execute()
```

EXPECTED RESULT

Target has one row per order_id; expected updates/inserts occurred; rerunning the same delta is materially idempotent.

WHY IT WORKED

Appending corrections creates duplicate current-state entities; an unsafe merge predicate can update/delete too much.

BREAK / FIX / DEBUG

If merge output surprises you, stop and validate source uniqueness, target uniqueness and predicate cardinality before rerunning.



C3-04-L10 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Merging before proving source/target key uniqueness.
- Using a predicate that does not match target grain.
- Treating MERGE metrics as the only validation.
- Rerunning without checking idempotent current-state results.

STUCK? RECOVERY

1) Target grain? 2) Source unique? 3) Predicate? 4) Metrics? 5) Key/count/amount check? 6) Rerun?

SMALL INDEPENDENT TASK

Apply a fresh correction to a different order and insert a new order with a different region.

VALIDATE

Target has one row per order_id; expected updates/inserts occurred; rerunning the same delta is materially idempotent.

THEN SHOW THE PROFESSIONAL VERSION

Build a fresh idempotent upsert and defend the merge predicate.

TEACH IT BACK

Why are append and upsert different current-state behaviors?

PROTECTED REVIEW + CHANGED RETRY

Save P10 first. Only then open R10. Close review and solve the changed version without reopening the answer. The original attempt remains in history.

EVIDENCE / MASTERY

MERGE code + metrics + independent reconciliation + rerun evidence.

MASTER only after the independent changed version passes validation and evidence is saved.



C3-04-L11 - Time travel and reproducible historical reads

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

"Current table looks right" is insufficient evidence when investigating when a bad change appeared or reproducing a historical report.

WHAT YOU LEARN

Read and compare prior table versions for audit/debug/reproducibility and explain the retention dependency of historical reads.

PREREQUISITE

C3-03 Gate PASS. Bridge warehousing/lakehouse concepts are assumed. Within C3-04, previous lesson mastery/remediation is expected. Parquet and transactional-table mechanics are taught from first principles here.

PICTURE IT IN YOUR HEAD

Picture labeled snapshots of a document. Version 5 lets you inspect what the table looked like before a change, but only while the files required to reconstruct version 5 still exist.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
time travel	Reading an earlier committed table version/snapshot.
version	Monotonic identifier of committed table state.
retention	Policy controlling how long historical files/state are preserved.
reproducibility	Ability to recreate an earlier analytical result from declared inputs/state.

USEFUL SOURCE / VIDEO

REQUIRED - [Delta Lake quick start - time travel](#)

Use versioned historical-read example and connect it to transaction-log state.

RECOMMENDED VIDEO - [Databricks - Diving into Delta Lake: Unpacking the Transaction Log](#)

Focus the time-travel/version-state chapters. External media remains optional.



C3-04-L11 - Follow with me once

Page B - engineering idea, tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

A versioned table can reconstruct older committed states while required data/log files remain available. Time travel is valuable for audit, rollback reasoning and reproducible analysis, but retention/vacuum can intentionally make old versions unreadable.

TINY WORKED EXAMPLE

Compare version before the day-2 merge/evolution with current version. Identify exactly which key values and schema fields changed.

FOLLOW WITH ME ONCE

List history/version -> load prior -> load current -> compare schema/keys/measures -> explain change -> connect to retention.

```
dt = DeltaTable(path)
current = dt.version()
old = DeltaTable(path, version=max(0, current-1))
# compare exact schema/keys/measures; save observed version IDs
```

EXPECTED RESULT

Historical read is real runtime evidence; version difference is specific; retention implication is stated.

WHY IT WORKED

"Current table looks right" is insufficient evidence when investigating when a bad change appeared or reproducing a historical report.

BREAK / FIX / DEBUG

If an old version is unreadable after cleanup, do not invent the past state. Record retention/recovery failure and restore from approved backup/history if available.



C3-04-L11 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Assuming old versions remain readable forever.
- Claiming historical state without actually reading it.
- Vacuuming files before understanding the recovery window.
- Comparing versions without naming exact schema/key/metric differences.

STUCK? RECOVERY

1) Current version? 2) Prior version? 3) Schema diff? 4) Row/key diff? 5) Retention?

SMALL INDEPENDENT TASK

Create another table change, then compare the two newest versions without using the earlier comparison text.

VALIDATE

Historical read is real runtime evidence; version difference is specific; retention implication is stated.

THEN SHOW THE PROFESSIONAL VERSION

Use a fresh historical comparison to locate and explain a table change.

TEACH IT BACK

How does time travel support both debugging and reproducibility?

PROTECTED REVIEW + CHANGED RETRY

Save P11 first. Only then open R11. Close review and solve the changed version without reopening the answer. The original attempt remains in history.

EVIDENCE / MASTERY

Version comparison + changed-history transfer.

MASTER only after the independent changed version passes validation and evidence is saved.



C3-04-L12 - Medallion/layered architecture and table contracts

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Without contracts, layered architecture becomes redundant copying and hides where quality/business rules actually belong.

WHAT YOU LEARN

Design bronze/silver/gold as explicit contracts rather than decorative layer names, with source evidence, validated semantics and consumer-specific outputs.

PREREQUISITE

C3-03 Gate PASS. Bridge warehousing/lakehouse concepts are assumed. Within C3-04, previous lesson mastery/remediation is expected. Parquet and transactional-table mechanics are taught from first principles here.

PICTURE IT IN YOUR HEAD

Picture a food-processing line. Bronze preserves source evidence, silver establishes reusable trusted ingredients, and gold prepares a consumer-specific dish. The names matter only when each boundary has a real contract.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
bronze	Source-evidence/raw landing contract.
silver	Validated reusable semantic/current-state contract.
gold	Consumer-oriented serving/aggregate contract.
table contract	Declared grain/schema/quality/owner/refresh/replay/change policy.

USEFUL SOURCE / VIDEO

RECOMMENDED - [Delta Lake documentation](#)

Use lakehouse/table semantics as architecture reference; bronze/silver/gold contract design remains course-built and vendor-neutral.



C3-04-L12 - Follow with me once

Page B - engineering idea, tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

Bronze commonly preserves source evidence and ingestion metadata; silver establishes validated types/keys/semantics; gold serves defined consumer grains/metrics. These names are useful only when allowed inputs, output grain, quality, owner and refresh/recovery behavior are explicit.

TINY WORKED EXAMPLE

Define orders_bronze, orders_silver_current and sales_gold_daily. State each grain, required keys, accepted/quarantined rules, consumers and replay source.

FOLLOW WITH ME ONCE

Source evidence -> validated reusable state -> consumer-specific serving; annotate checks and ownership at each transition.

EXPECTED RESULT

Every layer has a distinct purpose/grain; no source evidence is destroyed; consumer dependency is explicit.

WHY IT WORKED

Without contracts, layered architecture becomes redundant copying and hides where quality/business rules actually belong.

BREAK / FIX / DEBUG

If every layer is simply a copy, remove redundant layers or add a real contract boundary.



C3-04-L12 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Making bronze/silver/gold copies with no distinct purpose.
- Destroying raw/source evidence during cleanup.
- Putting consumer-specific logic into reusable silver without justification.
- Using layer names without grain/quality/owner/replay contracts.

STUCK? RECOVERY

1) Preserve? 2) Validate? 3) Serve whom? 4) Grain? 5) Quality? 6) Replay?

SMALL INDEPENDENT TASK

Design layers for support events where gold is a daily SLA dashboard aggregate.

VALIDATE

Every layer has a distinct purpose/grain; no source evidence is destroyed; consumer dependency is explicit.

THEN SHOW THE PROFESSIONAL VERSION

Design fresh layered contracts and remove an unnecessary layer if it adds no value.

TEACH IT BACK

Why is "bronze/silver/gold" not an architecture by itself?

PROTECTED REVIEW + CHANGED RETRY

Save P12 first. Only then open R12. Close review and solve the changed version without reopening the answer. The original attempt remains in history.

EVIDENCE / MASTERY

Three contracts + fresh domain transfer.

MASTER only after the independent changed version passes validation and evidence is saved.

C3-04-L13 - Optimization, retention and maintenance strategy

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Aggressive cleanup can destroy required recovery history; no cleanup creates unnecessary storage cost. Reliability requires explicit policy.

WHAT YOU LEARN

Operate a table lifecycle that balances file layout, metadata, time-travel/recovery requirements, cleanup safety and cost.

PREREQUISITE

C3-03 Gate PASS. Bridge warehousing/lakehouse concepts are assumed. Within C3-04, previous lesson mastery/remediation is expected. Parquet and transactional-table mechanics are taught from first principles here.

PICTURE IT IN YOUR HEAD

Picture housekeeping with two different jobs: reorganizing boxes to make access faster and throwing old boxes away. Compaction/optimization rewrites layout; vacuum/retention can permanently remove old files. Cleanup needs a recovery-window safety check.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
optimize	Maintenance operation that improves physical file layout.
vacuum	Removal of obsolete unreferenced files subject to retention rules.
dry run	Preview of destructive cleanup candidates before deletion.
recovery window	Historical period that must remain recoverable/reproducible.

USEFUL SOURCE / VIDEO

REQUIRED - [delta-rs - Managing a Table](#)

Use vacuum dry-run/history/maintenance API semantics.

REQUIRED - [delta-rs small-file compaction](#)

Use optimize vs vacuum distinction and safe maintenance workflow.



C3-04-L13 - Follow with me once

Page B - engineering idea, tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

Compaction/optimization improves file layout; vacuum/retention deletes obsolete historical files. These are different operations with different safety gates. delta-rs vacuum uses a dry-run by default and old versions may become invalid after files are removed. Maintenance should be evidence-triggered and followed by reconciliation.

TINY WORKED EXAMPLE

Write a runbook that measures file health, proposes compaction, dry-runs cleanup, checks the required recovery window, obtains approval for destructive cleanup and validates current truth afterward.

FOLLOW WITH ME ONCE

Measure -> diagnose -> compact if needed -> validate -> dry-run retention -> check recovery policy -> approve -> cleanup -> validate.

```
dt = DeltaTable(path)
# inspect candidates first
print(dt.vacuum(dry_run=True))
# destructive cleanup is NOT required for this lesson
```

EXPECTED RESULT

No destructive cleanup occurs without recovery-window check; maintenance has before/after integrity controls.

WHY IT WORKED

Aggressive cleanup can destroy required recovery history; no cleanup creates unnecessary storage cost. Reliability requires explicit policy.

BREAK / FIX / DEBUG

If retention conflicts with a required audit window, extend retention or create an approved alternate recovery mechanism before deleting files.



C3-04-L13 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Vacuuming to save space without recovery approval.
- Calling optimize a deletion operation.
- Skipping dry run before destructive cleanup.
- Using no before/after integrity controls or measurable trigger.

STUCK? RECOVERY

1) File health? 2) Compaction? 3) History need? 4) Dry-run? 5) Approval? 6) Reconcile?

SMALL INDEPENDENT TASK

Design a policy for a regulated table requiring 90-day reproducibility and compare it with a low-risk append-only staging table.

VALIDATE

No destructive cleanup occurs without recovery-window check; maintenance has before/after integrity controls.

THEN SHOW THE PROFESSIONAL VERSION

Defend a fresh maintenance/retention policy with safety gates and measurable triggers.

TEACH IT BACK

What can vacuum save, and what can it permanently remove?

PROTECTED REVIEW + CHANGED RETRY

Save P13 first. Only then open R13. Close review and solve the changed version without reopening the answer. The original attempt remains in history.

EVIDENCE / MASTERY

Runbook + dry-run design + changed-policy transfer.

MASTER only after the independent changed version passes validation and evidence is saved.

Mastery checklist + revision quick reference

- ☐ Every lesson: guided rebuild completed, genuine independent attempt saved, validation recorded.
- ☐ Protected review opened only after the attempt; changed retry solved after closing the review.
- ☐ Explain-back: business/engineering purpose, grain or state, one failure mode, one recovery step, one validation check.
- ☐ Mini-Lab / integrated drill completed with reproducible evidence.
- ☐ Gate A attempted independently. If NEEDS_REPAIR: review viewed, targeted repair completed, changed micro-proof recorded, then a fresh B/C assigned explicitly.
- ☐ No unresolved critical failure, secret/private data leakage, fabricated live-run evidence or copied Gate solution.
- ☐ Evidence is saved with enough context for another person to reproduce or inspect it.

Revision quick reference

Wrong result / wrong grain	Return to validation + data contract / grain lesson; reproduce smallest failing case.
Works once, rerun breaks	Return to idempotence/state/checkpoint/recovery lesson; prove clean replay.
Cannot explain failure	Return to mental model + break/fix section; write the failure boundary in your own words.
Gate NEEDS_REPAIR	Open only its protected review, repair named weak competency, solve changed micro-proof, then assign fresh retest.
Tool/vendor changed	Use the current official source for syntax/UI, but keep the book's durable engineering contract and validation.